Semantic Prediction in Sparse Automotive Radar Data Using RadarScenes

Gaïa Mezaib Boti
Data and Modelling / Machine Learning Final Project

This report studies semantic classification from sparse automotive radar using the RadarScenes dataset. The project processes 50 driving sequences, builds point-level and track-level feature tables, evaluates supervised classifiers with sequence-aware validation, and interprets results using feature importance, principal component analysis, and confidence intervals. The central result is that point-level classification is difficult because one radar return contains limited semantic context. Aggregating detections belonging to the same tracked object substantially improves performance: point-level Random Forest reaches 0.3714 accuracy and 0.3172 macro F1, while track-level XGBoost reaches 0.7757 accuracy and 0.5370 macro F1 under repeated sequence-level evaluation. The main scientific conclusion is therefore not simply that a boosted-tree model performs well, but that radar semantic prediction becomes substantially more reliable when sparse detections are interpreted as object-level temporal evidence rather than as isolated points.

1 Introduction

Automated driving systems need perception methods that remain useful when visual sensing is degraded. Automotive radar is attractive because it directly measures range and radial velocity and is comparatively robust to lighting and weather. However, radar is sparse: a scene is represented by reflection points rather than dense object silhouettes. Semantic prediction from radar alone is therefore challenging.

The research question is: *can radar measurements alone be used to predict semantic object classes, and does object-level aggregation improve performance?* The project evaluates this question on RadarScenes (Schumann et al., 2021). The work is not only a model benchmark; it is also an investigation of the data hierarchy, feature meaning, validation protocol, and limitations needed for a defensible machine-learning study.

The motivation is practical as well as methodological. In an autonomous-driving stack, it is useful to know not only that an object exists, but also what kind of object it is. Passenger cars, trucks, bicycles, and pedestrians do not have the same motion patterns, safety margins, or planning consequences. Cameras and lidar often provide richer shape information, but they can be affected by illumination, glare, fog, rain, or snow. Radar complements these sensors because it observes distance and velocity directly and can remain informative in conditions where visual appearance is unreliable.

At the same time, radar semantic prediction is harder than visual image classification. A radar scan contains sparse electromagnetic reflections rather than texture, colour, and object boundaries. One physical object may generate only a few radar points, and different objects can generate similar local returns. The project therefore compares two formulations: classifying one detection, and classifying an aggregated tracked object.

The project turns a large raw radar dataset into two defensible learning tasks and shows that the object-level task is substantially more informative. It is a radar-only study: camera images are used for context only, not as model inputs.

Report roadmap. The next sections define the RadarScenes hierarchy, explain the point-level and track-level feature tables, analyse feature structure with PCA and importance methods, describe the supervised models and evaluation protocol, and finally interpret why track-level aggregation improves the semantic prediction task.

2 Dataset and Problem Definition

2.1 RadarScenes hierarchy

RadarScenes contains 158 driving sequences. This project processes 50 sequences, which was com-

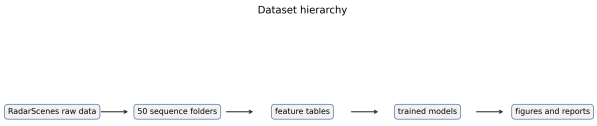


Figure 1: *Project data lineage from RadarScenes sequences to feature tables and machine-learning outputs.*

Table 1: *Canonical counts for the 50-sequence subset used in the final project.*

Quantity	Count
Sequences used	50
Scenes	227,089
Radar detections	28,749,044
Tracked detections	1,037,232
Untracked detections	27,711,812
Track-level rows	2,175
Camera images	10,971
Odometry samples	438,809

putationally feasible while still moving beyond the original single-sequence prototype. Each sequence folder contains radar and odometry measurements in `radar_data.h5`; camera images are available for visual context only and are not model inputs.

Each sequence is one continuous driving recording. Within a sequence, RadarScenes defines scenes, radar detections, odometry measurements, semantic labels, and object tracks. The odometry data describes the ego vehicle motion using position, orientation, velocity, and yaw rate. These quantities are important for interpreting radar measurements because a raw radial velocity includes the effect of the ego vehicle motion. The dataset also contains camera images, but this project uses them only for qualitative understanding and presentation context. No camera pixels are used as classifier features.

Figure 1 summarizes the data lineage used throughout the project, and Table 1 gives the canonical counts for the processed 50-sequence subset.

The project hierarchy is defined as a compact sequence of units:

Sequence: one driving recording.
Scene: one radar measurement from one sensor at one timestamp.
Detection: one radar return, also called one radar point.
UUID: unique identifier for one detection.
Track ID: identifier linking detections from the same dynamic object over time.
Aggregation: grouping detections into track-level object features.
ML: point-level and track-level semantic classification.

This project does not track the exact same radar point through time. It aggregates multiple detections sharing an object-level `track_id`, when such a dynamic-object track is available.

The difference between a scene and a detection is especially important. A scene is a measurement event associated with one radar sensor at one timestamp; it can contain many detections. A detection is the individual radar return with its own range, azimuth, RCS, velocity, position, `uuid`, and label. In simple terms, a sequence is a drive, a scene is one radar snapshot from one sensor, and a detection is one point inside that snapshot.

RadarScenes uses four automotive radar sensors mounted at different physical positions and orientations around the vehicle. Because the sensors do not occupy the same location, the same object can have different sensor-coordinate ranges and azimuths depending on which sensor observed it. The project therefore uses both sensor-coordinate variables, such as `range_sc` and `azimuth_sc`, and common coordinate variables, such as `x_cc` and `y_cc`, which locate detections in the car coordinate system. The four sensors measure at different timestamps rather than forming one perfectly simultaneous global scan, so sequence-aware validation is preferable to random point-level splitting.

2.2 Sensors, labels, and object identity

The four-sensor setup matters because radar perception depends strongly on viewpoint. A pedestrian, car, or bicycle may be visible to one sensor and partially hidden from another. Sensor-coordinate range and azimuth are therefore local to the observing sensor. Car-coordinate variables provide a common reference frame centred on the ego vehicle, making detections from different sensors more comparable. This is why both coordinate systems are retained: the sensor frame preserves how the radar measured the return, while the car frame preserves where the return lies around the vehicle.

The prediction target is `label_id`, a multiclass semantic label such as passenger car, truck, bicycle, pedestrian, pedestrian group, other dynamic object, or static environment. Most detections are untracked, largely because static environment detections do not belong to dynamic object tracks. Track identifiers are useful but not perfect: occlusion, stopped motion, or tracking switches can create new identities.

This identity structure is a common source of confusion. A `uuid` and a `track_id` do not answer the same question. The `uuid` asks: "which exact detection row is this?" The `track_id` asks: "which dynamic object does this detection belong to?" Multiple UUIDs can therefore share one track ID. Static environment points usually have no track ID because they are not moving objects being tracked across time.

The large difference between the number of detections and the number of track-level rows follows di-

rectly from this hierarchy. The 50-sequence subset contains 28,749,044 point-level detections, but only 1,037,232 of them are associated with dynamic-object tracks. These tracked detections are grouped into 2,175 object-level rows. This does not mean that the dataset contains only 2,175 physical objects overall. It means that the final track-level feature table contains 2,175 usable tracked dynamic-object identities within the processed subset and filtering choices.

Identity clarification. UUID identifies one radar detection. `track_id` links many detections assigned to the same dynamic object. The project aggregates by track identity but never uses the raw track identifier as a model feature.

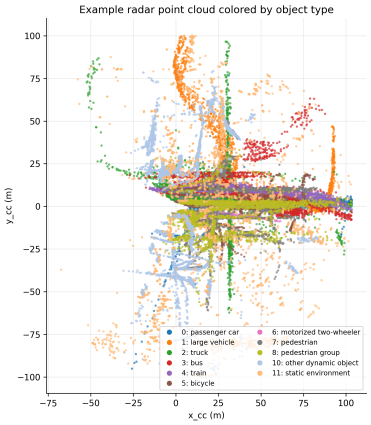


Figure 2: *Example radar point cloud coloured by object label. Radar observations are sparse and class-overlapping at the point level.*

3 Data Processing and Feature Engineering

3.1 Point-level representation

The point-level table has one row per radar detection. Raw input features include range, azimuth, radar cross section (RCS), radial velocity, ego-motion-compensated radial velocity, car-coordinate position, sequence-coordinate position, and sensor identifier. Engineered features include radius in car coordinates, absolute velocities, trigonometric angle encodings, the product `x_cc*y_cc`, and RCS divided by range.

The raw features have direct physical meaning. `range_sc` is the radial distance from the observing radar sensor to the detection in the sensor coordinate system. `azimuth_sc` is the horizontal bearing angle of the detection relative to that sensor. `rcs`, or radar cross section, is a reflectivity measure; it is influenced by the object's material, size, orientation, range, and multipath effects. `vr` is the measured radial velocity, while `vr_compensated` corrects this velocity for ego-vehicle motion. The variables `x_cc` and `y_cc` express the point position in a common car coordinate system, and `x_seq` and `y_seq` express it in the sequence coordinate system. Finally, `sensor_id` records which radar sensor produced the detection.

The engineered point-level features were added to make simple physical relationships easier for the models to use. `radius_cc` summarizes distance from the ego vehicle in common car coordinates. Absolute velocity features capture motion magnitude regardless of sign, which is useful because approaching and receding objects may both be semantically informative. Sine and cosine encodings of azimuth avoid the discontinuity that occurs when angles wrap around. `xy_product` captures a simple spatial interaction between longitudinal and lateral position. `rcs_per_range` provides a rough normalization of reflectivity by distance-related effects, although it is not a complete physical radar calibration.

The point-level feature groups can be interpreted as follows:

Table 2: *Point-level and track-level feature construction.*

Level	Feature representation
Point	`range_sc`, `azimuth_sc`, `rcs`, `vr`, `vr_compensated`, `x_cc`, `y_cc`, `x_seq`, `y_seq`, `sensor_id`, plus seven engineered features.
Track	Group by `sequence_id`, cleaned `track_id`, `label_id`, and label name; compute counts, means, standard deviations, min/max envelopes, duration, sensor coverage, density, and spatial spans.

- **Geometry:** range, azimuth, `x_cc`, `y_cc`, `x_seq`, `y_seq`, and radius describe where a return is located.
- **Motion:** `vr`, `vr_compensated`, and their absolute versions describe radial movement.
- **Reflectivity:** RCS and range-normalized RCS describe how strongly the target reflects radar energy.
- **Sensor context:** `sensor_id` records which radar observed the detection.
- **Angular encoding:** sine and cosine features avoid discontinuity in the azimuth angle.

The sparsity of the point-level representation is visible in Figure 2. Table 2 summarizes the distinction between the point-level and track-level feature tables.

3.2 Track-level aggregation

Track-level aggregation is crucial. For each dynamic object track, the pipeline computes `num_points`, `duration_seconds`, velocity statistics, RCS statistics, spatial spread, `number_of_sensors_seen`, `point_density`, and related features. The feature `num_points` should not be interpreted as pure object size: it mixes object size, observation duration, distance, visibility, reflectivity, aspect angle, sensor coverage, and tracking behavior.

The aggregation key is the cleaned dynamic-object identity together with sequence and label information. In practice, detections are grouped by `sequence_id`, cleaned `track_id`, `label_id`, and label name. Within each group, the pipeline computes descriptive statistics such as means, standard deviations, minima, maxima, spatial spans, and duration. For example, `mean_vr_compensated` summarizes the object's average ego-motion-corrected radial velocity over its track, while `std_rcs` summarizes how variable the object's reflectivity was during observation. Spatial envelope features such as `max_x_cc` and coordinate spans describe the region occupied by the track relative to the vehicle.

This object-level representation is not a manual tracking algorithm written from scratch. RadarScenes provides the `track_id`; the project uses it to build a machine-learning table. The model is therefore asked whether statistics of an annotated track are more predictive of semantic class than isolated point-level measurements.

The track-level features can be grouped into object descriptors:

- **Temporal persistence:** duration and number of detections describe how long and how often an object was observed.
- **Motion profile:** means and standard deviations of velocity summarize how the object moved during the track.
- **Spatial envelope:** min/max coordinates, coordinate spans, and radius spans describe the object's occupied region.
- **Observation quality:** sensor coverage and density describe how much radar evidence supports the object.
- **Reflectivity statistics:** mean and variability of RCS summarize the object's radar response over time.

The aggregation idea can be summarized as:

$$\mathbf{x}^{(\text{track})} = \text{aggregate}\left(\{\mathbf{x}_i^{(\text{det})} : \text{trackID}(i) = t\}\right), \quad (1)$$

where detections sharing the same dynamic-object track are converted into one object-level feature vector. Equation (1) is the central difference between the two feature representations: it adds object context without using the raw identifier as a classifier input.

Class imbalance affects both training and evaluation. Figure 3 shows a point-level example from the Sequence 102 prototype stage: static-environment detections dominate the radar points, while dynamic classes are much smaller. This figure is illustrative and sequence-specific; it is not presented as the final 50-sequence class distribution. The final interpretation therefore relies on macro F1, repeated sequence-level validation, and clearly separated point-level and track-level results.

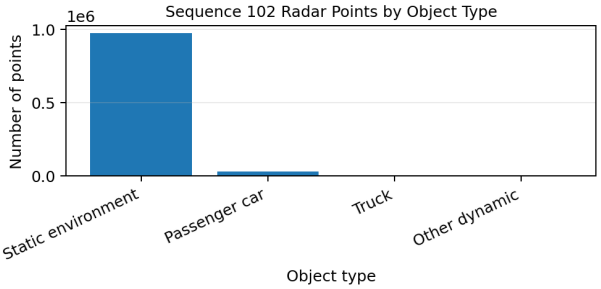


Figure 3: *Point-level class distribution for Sequence 102, used as a historical prototype example. Static-environment radar points dominate this sequence, which illustrates why accuracy alone can be misleading. This is not the final 50-sequence class distribution.*

4 Exploratory Feature Analysis

4.1 Why feature analysis was needed

Feature analysis used correlation, mutual information, Random Forest importance, permutation importance, and PCA. These methods answer different questions: correlation detects redundancy, mutual information measures dependence with the label, tree importance measures split usefulness, and permutation importance measures performance loss after shuffling a feature.

Correlation analysis is useful because several radar variables encode related geometry. For example, range, car-coordinate radius, and coordinate positions are not independent descriptions of the scene. High correlation does not automatically make a feature useless, but it warns that two columns may carry overlapping information. Mutual information complements this by looking at dependence between a feature and the label without assuming a linear relationship. This is important for radar because semantic classes may differ through nonlinear combinations of velocity, position, and density rather than through one simple threshold.

Random Forest feature importance and permutation importance were used as model-based checks. Tree importance identifies features that frequently reduce impurity in decision trees. Permutation importance asks a more direct predictive question: how much does model performance decrease when one feature is shuffled? These methods can disagree because they measure different things. The report therefore treats feature importance as evidence for interpretation, not as an absolute ranking of physical causality.

The main purpose of this analysis was to avoid treating the feature table as a black box. Radar features have physical meaning, and a defensible machine-learning project should explain why a model may rely on them. For example, velocity features are meaningful because different road users move differently. Spatial features are meaningful because objects appear in structured regions around the ego vehicle. RCS is meaningful

Table 3: *PCA thresholds for the 17 engineered point-level features.*

Cumulative variance target	Components needed
80%	7
90%	9
95%	10
99%	14

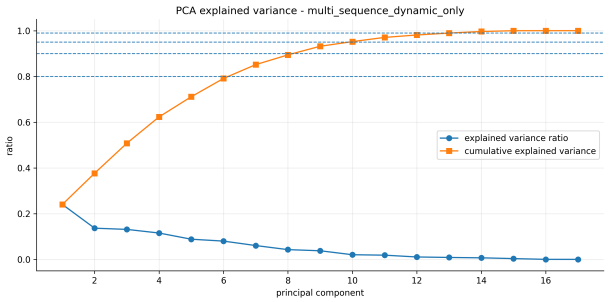


Figure 4: *Explained and cumulative variance for PCA on the engineered point-level radar feature set. PCA shows redundancy but does not measure classification accuracy.*

because it measures radar reflectivity, but it is noisy and viewpoint-dependent. Feature analysis therefore connects numerical performance to radar perception rather than only reporting scores.

4.2 PCA as a structure check

PCA was applied to the 17 engineered point-level radar features, not only to a smaller raw-feature baseline (Pearson, 1901). These features are `range_sc`, `azimuth_sc`, `rcs`, `vr`, `vr_compensated`, `x_cc`, `y_cc`, `x_seq`, `y_seq`, `sensor_id`, `radius_cc`, `abs_vr`, `abs_vr_compensated`, `azimuth_sin`, `azimuth_cos`, `xy_product`, and `rcs_per_range`. PCA was used for feature-structure interpretation, not as the final classifier pipeline.

After standardization, the PCA transformation can be written compactly as:

$$\mathbf{Z} = \mathbf{X}_{\text{standardized}}\mathbf{W}, \tag{2}$$

where $\mathbf{W}$ contains the principal directions. This equation is included to clarify the analysis step only; the final benchmark models are not presented as mandatory PCA-reduced pipelines. Equation (2) is therefore an analysis equation, not the definition of the final prediction pipeline.

The first components combine geometry, range, velocity, angular structure, and reflectivity. This confirms that radar features are not independent: several columns describe related physical phenomena. It does not imply that features should automatically be discarded, because variance preservation is different from label prediction.

The PCA thresholds in Table 3 also explain why the "14 principal components" statement is valid. PCA was not run on only seven raw variables. It was run on a larger 17-feature engineered point-level table, so up to 17 principal components are possible. The fact that 99% of variance requires 14 components indicates that the engineered feature set contains redundancy, but not so much redundancy that only two or three axes describe the data. In other words, the radar representation is structured but still multidimensional.

Figure 4 shows the explained-variance curve used to support these thresholds.

It is also important not to confuse PCA variance with classification performance. PCA is unsupervised: it summarizes directions of high feature variance without using the class labels. A component can explain large variance because it captures position or range differences, even if that variance does not strongly separate semantic classes. For this reason, PCA was used to understand the feature space and potential redundancy, while the final classifiers were trained on the verified feature tables rather than on a mandatory PCA-reduced pipeline.

PCA interpretation. The engineered feature set has redundancy, but PCA variance is not classification accuracy. Supervised models still determine which relationships are useful for predicting labels.

5 Machine Learning Methodology

5.1 Two supervised learning tasks

The supervised learning problem is to learn a function f from radar features X to semantic labels $y = \texttt{label_id}$. Two tasks are evaluated: point-level classification, where one sample is one detection, and track-level classification, where one sample is one aggregated dynamic object.

The supervised-learning objective is summarized as:

$$\hat{y} = f(\mathbf{x}), \tag{3}$$

where $\mathbf{x}$ is either a point-level radar feature vector or a track-level aggregated feature vector. Equation (3) is intentionally general because the project compares two different choices for $\mathbf{x}$.

The point-level and track-level tasks answer related but different questions. Point-level classification asks whether a single radar return contains enough information to infer a semantic label. Track-level classification asks whether a collection of detections belonging to the same dynamic object, summarized over time, is more informative. The second task has access to richer context, but it also uses a much smaller dataset because only detections with usable dynamic-object track IDs can be aggregated.

This distinction is essential for interpreting the final results. A point-level row is plentiful but weak: there

Table 4: *Compact comparison of the two learning tasks.*

Aspect	Point-level	Track-level
Sample	one radar detection	one tracked object
Data volume	many samples	fewer samples
Evidence	sparse local return	motion and spatial context
Final model	Random Forest	XGBoost
Accuracy	0.3714	0.7757
Macro F1	0.3172	0.5370

are millions of detections, but each one contains little semantic context. A track-level row is scarce but richer: there are only 2,175 rows in the final track table, but each row summarizes many detections from the same dynamic object. Therefore the project does not claim that the two tasks are directly identical. Instead, it compares how much predictive information is available at each representation level.

Figure 5 shows the full modelling flow, and Table 4 gives the compact version of the point-level versus track-level comparison.

5.2 Model families

Implemented models include Logistic Regression, Gaussian Naive Bayes, KNN, Linear SVM, RBF SVM, Random Forest, XGBoost, MLPClassifier, and an exploratory CNN. The report emphasizes Logistic Regression, KNN, Gaussian Naive Bayes, Random Forest, and XGBoost because these appear in the final benchmark outputs. The CNN is exploratory: it rasterizes radar tracks into small multi-channel grids and does not use camera images.

Logistic Regression provides a linear softmax baseline. It is useful because it tests whether classes are separable by approximately linear combinations of the features. KNN predicts a label from nearby training examples in feature space, which makes it intuitive but sensitive to feature scaling and local class imbalance. Gaussian Naive Bayes is a probabilistic baseline: it estimates a class-wise mean and variance for each feature and assumes conditional independence between features. This assumption is clearly simplified for radar data, but the model is still valuable because it gives a transparent lower-complexity comparison.

Random Forest and XGBoost are stronger nonlinear baselines. Random Forest averages many decision trees trained with randomness, reducing the instability of a single tree. XGBoost builds trees sequentially, where each new tree focuses on errors left by previous trees. These models can capture threshold effects, interactions, and heterogeneous feature scales more naturally than a purely linear model. In this project, that flexibility is important because semantic class may depend on combinations such as velocity magnitude, sensor coverage, spatial envelope, and density.

The model families therefore play different roles:

- **Linear baseline:** Logistic Regression tests whether simple decision boundaries are sufficient.
- **Distance baseline:** KNN tests whether nearby radar feature vectors tend to share labels.
- **Probabilistic baseline:** Gaussian Naive Bayes tests a simple class-conditional density assumption.
- **Tree ensembles:** Random Forest and XGBoost test nonlinear thresholds and feature interactions.
- **Exploratory neural models:** MLP and CNN experiments explore nonlinear representation learning without becoming the final claim.

Tree-based models are appropriate because radar classification is nonlinear. Motion, geometry, sensor coverage, point density, and RCS interact in ways that are not well described by one linear boundary. Random Forest uses many decision trees (Breiman, 2001); XGBoost uses boosted trees that iteratively correct previous errors (Chen and Guestrin, 2016). Gaussian Naive Bayes provides a small course-aligned probabilistic baseline by estimating class-wise Gaussian means and variances from training data.

The exploratory neural-network work was treated carefully. A multilayer perceptron and an exploratory CNN can be useful for testing whether learned nonlinear representations improve performance. However, the CNN is not presented as a final vision model and does not use camera images. It is better understood as a preliminary experiment on rasterized radar representations. The final report therefore avoids overclaiming deep-learning performance and focuses on the validated tabular benchmarks.

Methodological summary. The final conclusions rely on models that are simple enough to audit and strong enough to capture nonlinear radar feature interactions. This is why the report emphasizes tree ensembles and transparent baselines rather than presenting the exploratory CNN as the main result.

6 Evaluation Protocol

6.1 Sequence-level validation and leakage

Final claims use sequence-aware splitting whenever possible. This reduces leakage because detections from the same driving sequence are correlated. If samples from one sequence appeared in both train and test, a model could benefit from shared scene context rather than generalizing to unseen sequences.

This is one of the most important evaluation choices. A random point-level split can be too optimistic for radar data because neighbouring detections, timestamps, and scenes are not independent. If the same drive contributes samples to both training and testing, the model may learn sequence-specific conditions such as road layout, sensor context, or class distribution

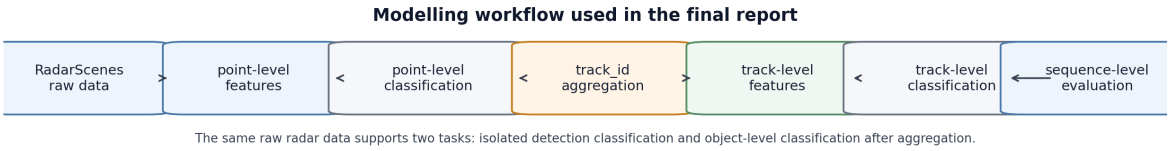


Figure 5: *Compact modelling workflow. Raw RadarScenes data is converted into point-level features, evaluated directly, and then aggregated with* `track_id` *into track-level features for object-level classification.*

rather than general semantic patterns. Sequence-level splitting is stricter: entire sequences are held out, so the test set better represents unseen driving recordings.

The final results distinguish three kinds of evidence. First, the point-level Random Forest benchmark reports the best full dynamic-only point-level result available from the project outputs. Second, the point-level repeated robustness experiment is explicitly labelled as a sampled/reduced stability check, not as a replacement for the full benchmark. Third, the track-level benchmark uses repeated sequence-level splits over the track table, providing mean results and confidence intervals for each model. This separation keeps the comparison honest.

The leakage assessment is:

- **No label leakage:** label columns are not used as input features.
- **No identity memorization:** raw `track_id` is not used as a model feature.
- **Sequence-aware testing:** held-out sequences reduce the risk that similar neighbouring scenes appear in both train and test.
- **Aggregation before modelling:** `track_id` is used to form object-level rows, then removed from the predictive feature set.

6.2 Metrics and uncertainty

Metrics are accuracy, macro precision, macro recall, and macro F1. For class k,

$$\mathrm{F1}_k = \frac{2\,\mathrm{precision}_k\,\mathrm{recall}_k}{\mathrm{precision}_k + \mathrm{recall}_k},$$

and macro F1 averages over classes equally:

$$\mathrm{MacroF1} = \frac{1}{K}\sum_{k=1}^{K}\mathrm{F1}_k. \tag{4}$$

Macro F1 is essential because accuracy can hide poor performance on rare classes. Equations (4) and the preceding class-wise F1 definition make explicit that the final macro score treats each class equally.

Confidence intervals quantify uncertainty. The point-level benchmark uses sequence-bootstrap confidence intervals on saved predictions. Track-level results use ten repeated sequence-level splits and report mean scores with 95% confidence intervals. A confidence interval is not a guarantee; it is a plausible range under the chosen resampling protocol.

Accuracy measures the fraction of correct predictions overall. Precision measures how reliable a predicted class is, while recall measures how many true examples of that class are recovered. F1 balances precision and recall. Macro F1 first computes class-wise F1 and then averages classes equally, so rare classes contribute as much to the final score as common classes. This is why macro F1 is lower than accuracy in many results: it exposes the difficulty of minority dynamic classes that would otherwise be hidden by aggregate accuracy.

A 95% confidence interval is included to communicate uncertainty rather than only a single number. Wide intervals indicate that performance depends strongly on which sequences are used for testing. This is not a failure of the project; it is valuable scientific information. RadarScenes contains different road layouts, object mixtures, weather conditions, and traffic situations, so sequence-to-sequence variability is expected. Reporting uncertainty makes the benchmark more defensible than presenting a single split as if it were exact.

The metrics answer complementary questions:

- **Accuracy:** how often the prediction is correct overall.
- **Precision:** when the model predicts a class, how often that prediction is reliable.
- **Recall:** how many true examples of a class the model finds.
- **Macro F1:** how well the model balances precision and recall across all classes equally.
- **Confidence interval:** how sensitive the metric is to the sampled test sequences or bootstrap resampling.

Evaluation summary. Sequence-level splitting estimates generalization to unseen drives. **Macro F1** prevents common classes from hiding poor rare-class performance. **Confidence intervals** show how sensitive results are to the tested sequences.

7 Results

7.1 Point-level benchmark

The strongest point-level dynamic-only benchmark is Random Forest, with 0.3714 accuracy and 0.3172 macro F1. Its 95% confidence intervals are wide, reflecting strong sequence sensitivity. Point-level repeated robustness was run on a sampled and balanced subset, so it is a stability check rather than a replacement for the full point-level benchmark.

The point-level result should be interpreted as a hard baseline rather than a failure. A single detection may correspond to a reflection from a wheel, bumper, pedestrian limb, bicycle frame, road boundary, or another partial surface. Such a point carries range, angle, velocity, and reflectivity, but not a complete object shape. Therefore many classes overlap in the point-level feature space. The Random Forest performs best among the point-level dynamic-only models because it can use nonlinear thresholds and feature interactions, but the macro F1 of 0.3172 shows that isolated detections are weak semantic evidence.

Table 5 reports the final point-level and track-level benchmark metrics, and Figure 7 visualizes the main performance gap.

The point-level sampled robustness experiment has a lower mean macro F1 of 0.2141. This result should not replace the full point-level benchmark because it uses a sampled/reduced setup. Its purpose is to test whether the ranking and difficulty remain visible under repeated sequence-level splits. The answer is yes: point-level classification remains unstable and difficult, especially for rare classes and sequence shifts.

Point-level result. Individual radar detections are informative but incomplete. They estimate local geometry and motion, but they do not reliably contain enough semantic context to separate all dynamic classes.

Figure 6 shows this overlap directly for the point-level Random Forest benchmark.

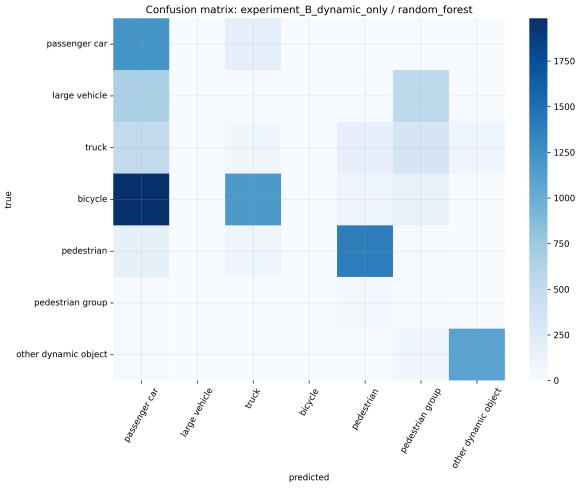


Figure 6: *Confusion matrix for the point-level dynamic-only Random Forest benchmark. It illustrates why point-level radar classification is difficult: several dynamic classes overlap strongly when each sample is only one sparse detection.*

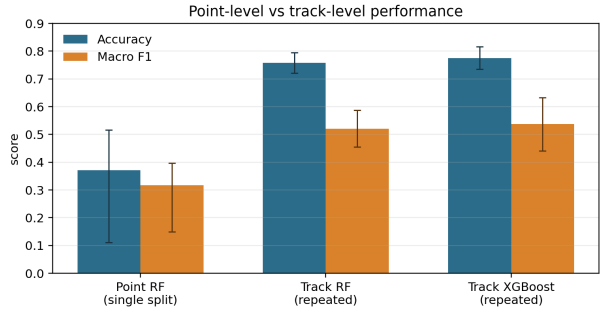


Figure 7: *Point-level versus track-level performance. Track-level aggregation substantially improves both accuracy and macro F1.*

7.2 Track-level benchmark

Track-level classification is substantially stronger. The best final result is track-level XGBoost, with mean accuracy 0.7757 and mean macro F1 0.5370 across ten sequence-level repetitions. Track-level Random Forest is close, reaching 0.7583 accuracy and 0.5211 macro F1. The gap between point-level and track-level performance is the main scientific result of the project.

The track-level results show that object context provides information that individual detections do not contain. XGBoost is strongest, but Random Forest is close, suggesting that the improvement is not only due to one specific algorithm. Both are tree-based models that can exploit nonlinear feature interactions. Logistic Regression performs worse but remains meaningful, indicating that some track-level signal is linearly separable. Gaussian Naive Bayes is weakest among the track-level benchmarks because its independence and Gaussian assumptions are too simple for aggregated radar features, which are correlated and often non-Gaussian.

The model ranking is physically plausible. Logistic Regression benefits from the aggregated features, but it is limited by linear boundaries. Gaussian Naive Bayes is more constrained because it assumes feature independence, while duration, number of points, spatial spread, and sensor coverage are naturally related. Random Forest and XGBoost perform better because they can combine compensated velocity, point density, and spatial envelope.

The top XGBoost track-level features are `mean_abs_vr_compensated`, `max_x_cc`, `number_of_sensors_seen`, `point_density`, and `mean_xy_product`. This is physically plausible. Compensated radial velocity removes ego-motion influence and better reflects object motion. Spatial features describe where the object appears relative to the vehicle. Sensor coverage indicates how persistently and from

Table 5: *Final point-level and track-level performance. Brackets show 95% confidence intervals where available.*

Setting	Model	Accuracy	Macro F1
Point-level dynamic-only, single split	Random Forest	0.3714 [0.1118, 0.5158]	0.3172 [0.1492, 0.3966]
Point-level sampled robustness	Random Forest	0.3162 [0.2462, 0.3863]	0.2141 [0.1732, 0.2550]
Track-level repeated benchmark	Logistic Regression	0.7113 [0.6971, 0.7255]	0.4167 [0.3703, 0.4631]
Track-level repeated benchmark	Gaussian Naive Bayes	0.5347 [0.4951, 0.5743]	0.2991 [0.2712, 0.3269]
Track-level repeated benchmark	Random Forest	0.7583 [0.7214, 0.7952]	0.5211 [0.4545, 0.5877]
Track-level repeated benchmark	XGBoost	0.7757 [0.7349, 0.8165]	0.5370 [0.4405, 0.6334]

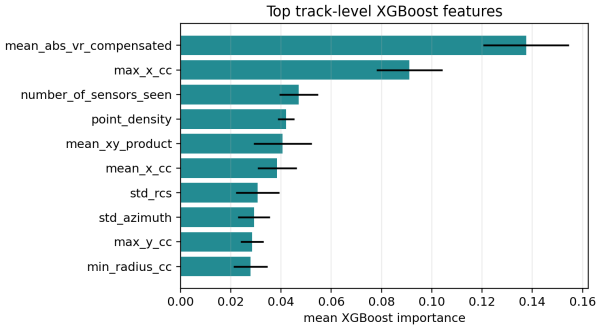


Figure 8: *Top repeated-split feature importances for the track-level XGBoost model. Motion, spatial envelope, sensor coverage, and density dominate.*

how many viewpoints an object is observed. Density captures compactness and observation persistence.

The feature importance result explains why track-level classification is not merely a numerical trick. The most important variables correspond to interpretable radar and driving concepts: motion, spatial envelope, sensor coverage, and point density. These are object-level properties, so they are naturally stronger after aggregation.

Figure 8 shows the repeated-split XGBoost feature-importance ranking that supports this interpretation.

Track-level result. The best result is not just a higher score. It shows that object-level temporal and spatial context carries semantic information that is mostly unavailable in a single radar return.

8 Discussion

8.1 Why track-level learning works

Point-level classification is difficult because one radar detection is a sparse and noisy measurement. A single point has no object outline, limited context, and ambiguous reflectivity. Different semantic classes can produce overlapping ranges, velocities, and RCS values at one instant.

Track-level aggregation changes the learning problem. Instead of asking the model to classify one return, the model sees an object profile over time. It can use motion consistency, duration, spatial spread, density, and multi-sensor visibility. This is why track-level classification outperforms point-level classification.

This finding aligns with the physical intuition of radar perception. Radar is excellent at measuring motion and distance, but it is not naturally a dense semantic sensor. A point-level detector observes fragments of objects. A track-level representation turns those fragments into a temporal signature. For example, a pedestrian track may have a different compensated-velocity profile, spatial spread, and persistence pattern than a passenger car track. A large vehicle may occupy a different spatial envelope and be visible to sensors differently than a bicycle. These patterns are not guaranteed for every instance, but they are more informative than a single reflection.

RCS is useful but not dominant. It depends on material, size, aspect angle, distance, occlusion, and multipath. The same object can reflect differently from different viewpoints, while different objects can have similar RCS. In this project, motion and geometry are more stable discriminative signals than raw reflectivity alone.

The importance of `mean_abs_vr_compensated` is particularly meaningful. Raw radial velocity mixes object motion with ego-vehicle motion, so the compensated version better describes the object's motion relative to the driving scene. This matters because semantic classes are partly defined by how they move: pedestrians, bicycles, cars, trucks, and groups of pedestrians tend to have different speed ranges and motion patterns. Velocity alone is not enough for perfect classification, but when combined with spatial and density features it becomes a strong signal.

Spatial features also outperform many RCS features because location and geometry summarize context. In driving scenes, object classes are not uniformly distributed everywhere around the vehicle. Pedestrians and bicycles often appear near sidewalks, crossings, or lateral regions, while vehicles are often aligned with lanes. The model should not be interpreted as learning road rules directly, but spatial coordinates help describe the observation geometry that radar sees. Sensor coverage adds another layer: an object seen by multiple radars or over a longer period usually has a more reliable radar signature.

The performance gain can therefore be understood through four object-level signals:

- **Temporal signal:** a track summarizes repeated observations rather than one instant.
- **Motion signal:** compensated velocity captures ob-

ject behaviour after removing ego-motion effects.
- **Spatial signal:** coordinate envelopes describe where and how the object appears around the vehicle.
- **Observation signal:** density and sensor coverage describe how much radar evidence supports the object.

8.2 Why the result is defensible

The improvement is not caused by using the raw `track_id` as a predictor. The split is sequence-level, no object appears in both train and test when sequences are disjoint, and `track_id` is used only to aggregate detections into statistics. The label columns are excluded from model features.

This leakage audit is essential. If `track_id` itself were used as a feature, the model could memorize identities rather than learning semantic structure. That is not the case here. Track IDs are used only before modelling to define which detections belong to the same object-level row. Once aggregation is complete, the classifier receives numerical summaries such as velocity means, spatial spans, density, and sensor counts. Because the split is sequence-level, tracks from a held-out sequence do not appear in the training set.

Sequence 102 should be understood historically. It was useful for debugging, visualization, and prototype development, but the final conclusions are based on the 50-sequence pipeline and sequence-aware evaluation. Older high Sequence 102 scores are not final benchmark results.

The final narrative is therefore coherent: the project began with individual detections, analysed their features, tested point-level classifiers, quantified uncertainty, and then used the dataset's dynamic-object identities to build track-level representations. The strongest conclusion is that object-level context is necessary for more reliable radar semantic prediction. This is a more scientifically useful result than reporting a single high score, because it explains what kind of information radar models actually need.

Final interpretation. Radar is strongest when it is allowed to express what it measures well: motion, distance, persistence, and multi-sensor visibility. These signals are weak as isolated points but substantially stronger when summarized over tracked objects.

9 Limitations and Future Work

9.1 Limitations

The project processes 50 of 158 RadarScenes sequences. This is enough for a defensible multi-sequence study, but not the full dataset. Rare classes remain challenging because the track table is much smaller than the point table and class counts are uneven. Radar-only semantic prediction remains intrinsically difficult because sparse radar signatures overlap.

The 50-sequence subset was a practical compromise. It allowed the project to move beyond a single prototype sequence while keeping feature extraction, validation, visualization, and report generation feasible. However, the remaining 108 sequences could contain additional weather, traffic, object, and road-layout variability. A final production-quality benchmark should process all 158 sequences and check whether the same feature rankings, PCA structure, and track-level improvements remain stable.

The track-level table is much smaller than the point-level table. This is expected because track-level learning requires dynamic-object `track_id` annotations, while most detections are untracked static environment points. The smaller table makes object-level learning more interpretable, but it also limits rare-class evaluation. Classes such as trains, buses, motorized two-wheelers, and large vehicles have very few track-level examples in the processed subset, so their class-specific scores should not be overinterpreted.

Camera images are used only for context and timestamp-aligned visualization, not as classifier inputs. No calibrated radar-to-camera projection is claimed. The CNN experiment is exploratory and uses rasterized radar tracks, not camera images. Track IDs are useful dynamic-object identifiers, but they can reset or switch in difficult tracking situations.

The project also does not claim to solve temporal tracking from scratch. It uses RadarScenes track IDs when they are available. This is appropriate for evaluating object-level semantic prediction, but it means that the quality and assumptions of the dataset's tracking annotations matter. If an object is occluded, stops for a long time, or is affected by a tracking switch, the assigned identity may change. Therefore `track_id` improves object context but does not guarantee perfect real-world identity forever.

The main limitations are:

- **Dataset coverage:** only 50 of 158 RadarScenes sequences were processed.
- **Class imbalance:** rare dynamic classes remain difficult to evaluate reliably.
- **Radar sparsity:** different classes can produce overlapping sparse radar signatures.
- **Track dependence:** track-level learning requires available and reliable dynamic-object track IDs.
- **No image fusion:** camera images are not used as model inputs.
- **Exploratory CNN:** rasterized radar CNN results are not treated as the final benchmark.

9.2 Future work

Future work should process all 158 sequences, improve temporal modelling, tune hyperparameters more systematically, evaluate calibrated radar-camera fusion,

and test sequence models, graph neural networks, or transformers over tracked objects and their interactions. Rare-class robustness and uncertainty calibration are also natural next steps.

Further work could also evaluate class-balanced training losses, hierarchical labels, and calibrated probability outputs. A hierarchy could distinguish static versus dynamic objects first and then classify dynamic subclasses. Uncertainty calibration would be valuable because an autonomous system should know when a radar-only semantic prediction is unreliable. Finally, graph-based methods could model relationships between nearby tracks, such as groups of pedestrians or vehicles moving together, which are not fully captured by independent tabular rows.

The most realistic next step is not to replace the current pipeline entirely, but to extend it. A stronger final system could keep the interpretable track-level features, add sequence models for temporal behaviour, and then compare radar-only and radar-camera fusion under the same sequence-level validation protocol. This would preserve the scientific discipline of the current project while testing whether richer temporal or multimodal information improves rare-class performance.

10 Conclusion

The project shows that radar-only semantic prediction is possible but difficult at the level of individual detections. A single radar point is a sparse reflection, not a complete object description, which explains the relatively low point-level macro F1. The most important finding is that object-level context matters: aggregating detections into tracked objects gives the model motion, spatial structure, sensor coverage, density, and persistence.

The final track-level XGBoost model is not only numerically strongest; it is strongest for a physically meaningful reason. The features it relies on most are linked to compensated motion, spatial envelope, multi-sensor observation, and density. These are exactly the cues radar can provide reliably. The project therefore supports a clear conclusion: sparse radar becomes more semantically informative when detections are connected over time into object-level representations.

11 Credits

11.1 Authors

Vel (`vel@LaTeXTemplates.com`) and Frits Wenneker are the authors of the Wenneker article template, downloaded from `http://www.LaTeXTemplates.com`. The template formatting was preserved; only `main.tex` and supporting report assets were edited for this project.

11.2 License

The Wenneker template is distributed under CC BY-NC-SA 3.0 (`http://creativecommons.org/licenses/by-nc-sa/3.0/`).

Bibliography

Breiman, Leo (2001). “Random Forests”. In: *Machine Learning* 45.1, pp. 5–32.

Chen, Tianqi and Carlos Guestrin (2016). “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.

Pearson, Karl (1901). “On Lines and Planes of Closest Fit to Systems of Points in Space”. In: *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science* 2.11, pp. 559–572.

Schumann, Ole et al. (2021). “RadarScenes: A Real-World Radar Point Cloud Data Set for Automotive Applications”. In: *IEEE International Conference on Intelligent Transportation Systems*.